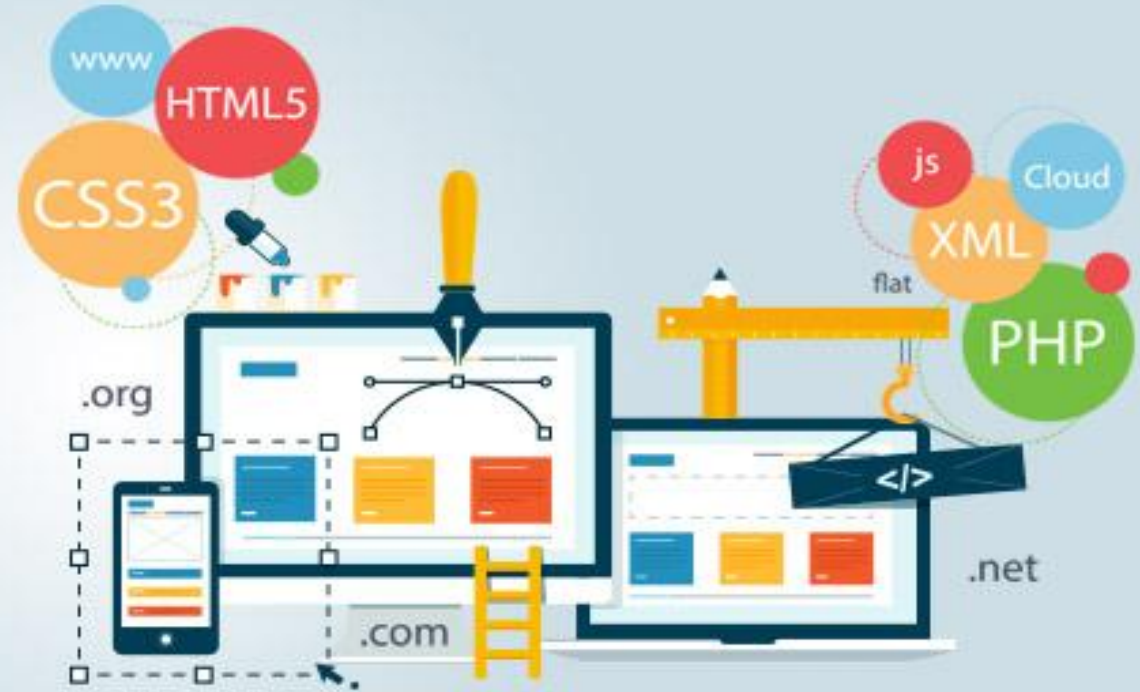




Facultad de Ingeniería –UNLPam  
General Pico – La Pampa – Argentina

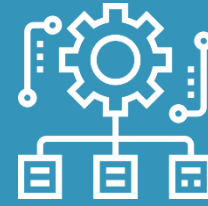
# PHP y MySql - AJAX

Lic. Gustavo Lafuente

# AJAX

## Programación Orientada a la Web

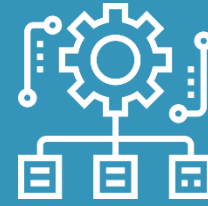
# AJAX



El término **AJAX** es un acrónimo de *Asynchronous JavaScript + XML*, que se puede traducir como "JavaScript asíncrono + XML".

“ **Ajax** no es una tecnología en sí mismo (no es un lenguaje de Programación). En realidad, se trata de varias tecnologías independientes que se unen de formas nuevas y sorprendentes.”

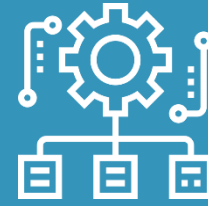
# AJAX



AJAX es una técnica de desarrollo web para crear aplicaciones interactivas o **RIA** (*Rich Internet Applications*).

Estas aplicaciones se ejecutan en el cliente, es decir, en el navegador de los usuarios mientras se mantiene la comunicación asíncrona con el servidor en segundo plano. De esta forma es posible realizar cambios sobre las páginas sin necesidad de recargarlas, mejorando la interactividad, velocidad y usabilidad en las aplicaciones.

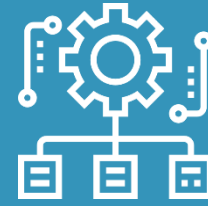
# AJAX



Ajax usa una metodología **asíncrona**, en el sentido de que los datos adicionales se solicitan al servidor y **se cargan en segundo** plano sin interferir con la visualización ni el comportamiento de la página.

Funciona en múltiples plataformas y utilizable en muchos sistemas operativos y navegadores dado que está basado en estándares abiertos.

# AJAX - Tecnologías



**XHTML** y **CSS**, para crear una presentación basada en estándares.

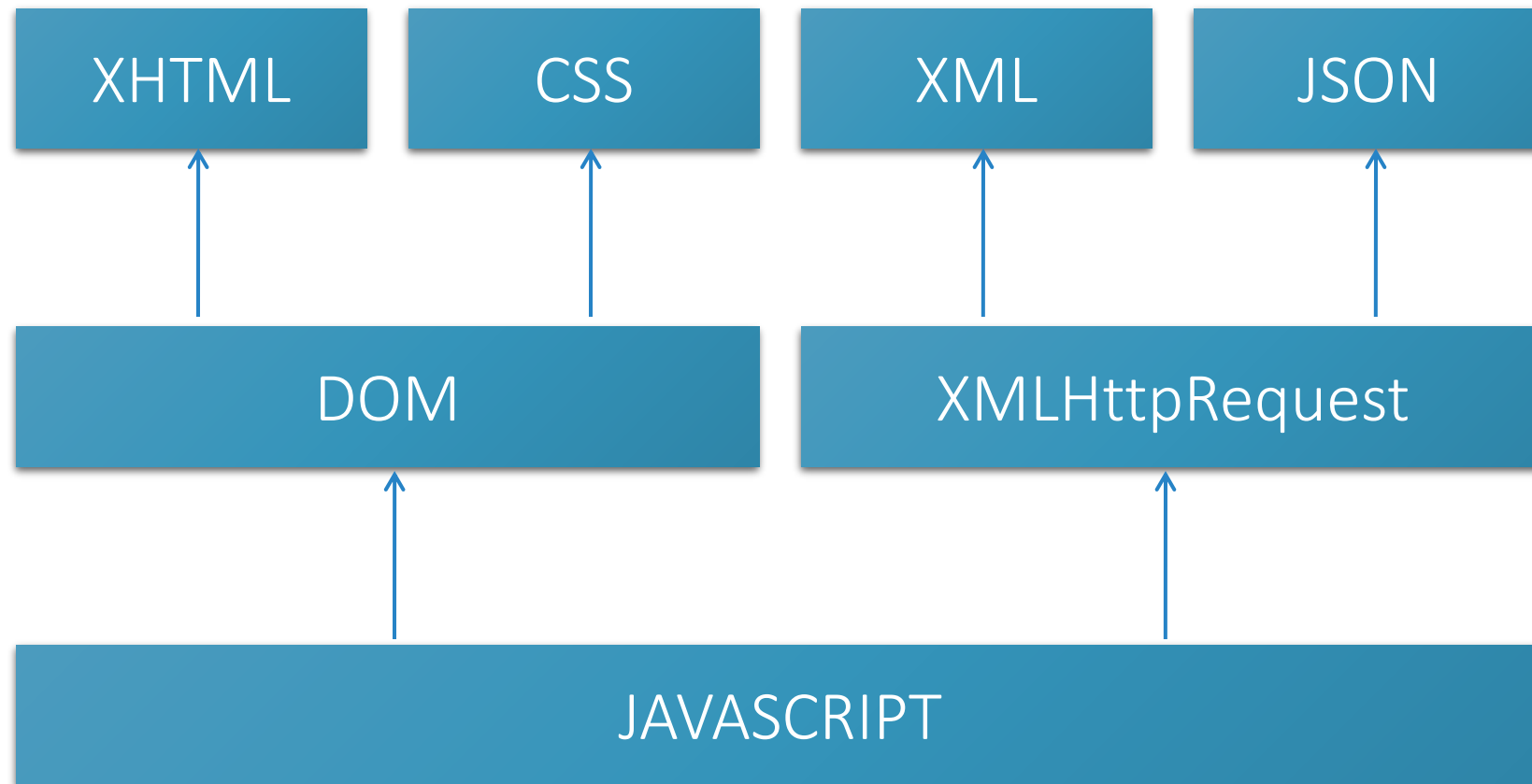
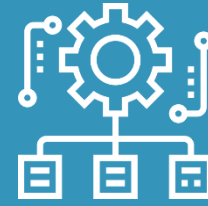
**DOM**, para la interacción y manipulación dinámica de la presentación.

**XML**, **XSLT** y **JSON**, para el intercambio y la manipulación de información. (también texto plano)

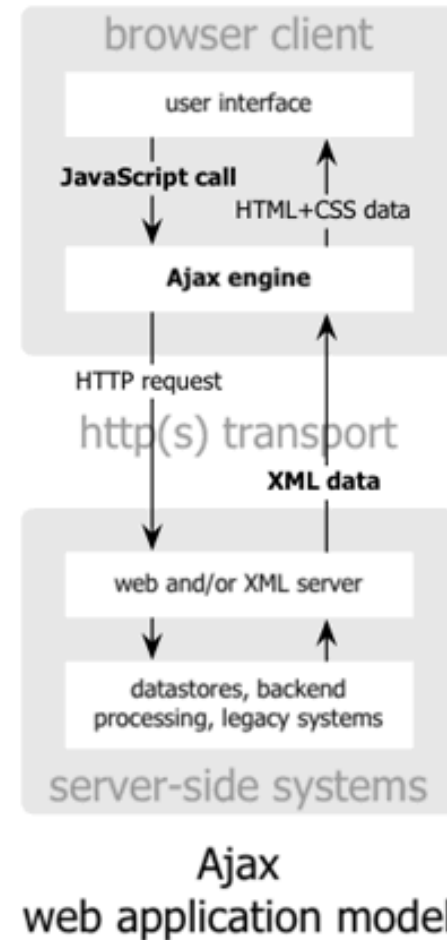
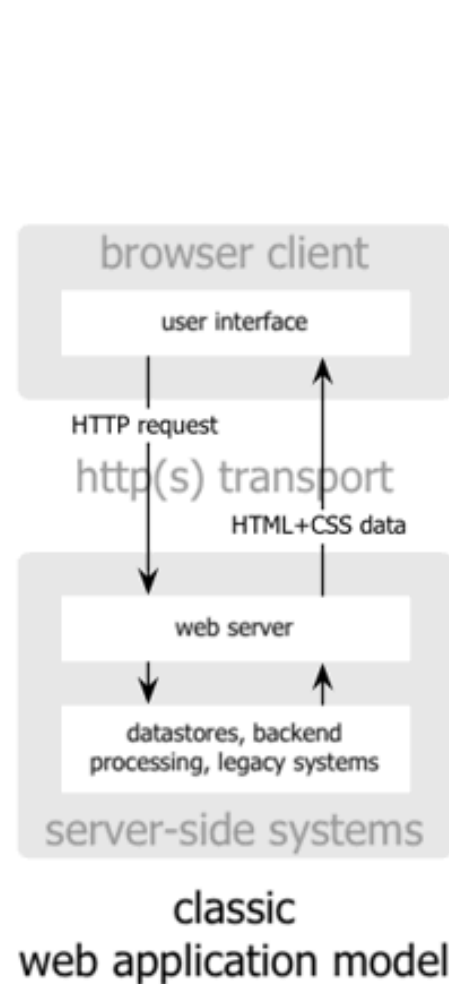
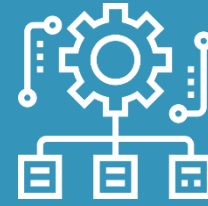
**XMLHttpRequest**, para el intercambio asíncrono de información.

**JavaScript**, para unir todas las demás tecnologías (ASP, PHP, etc).

# AJAX – Concepto general

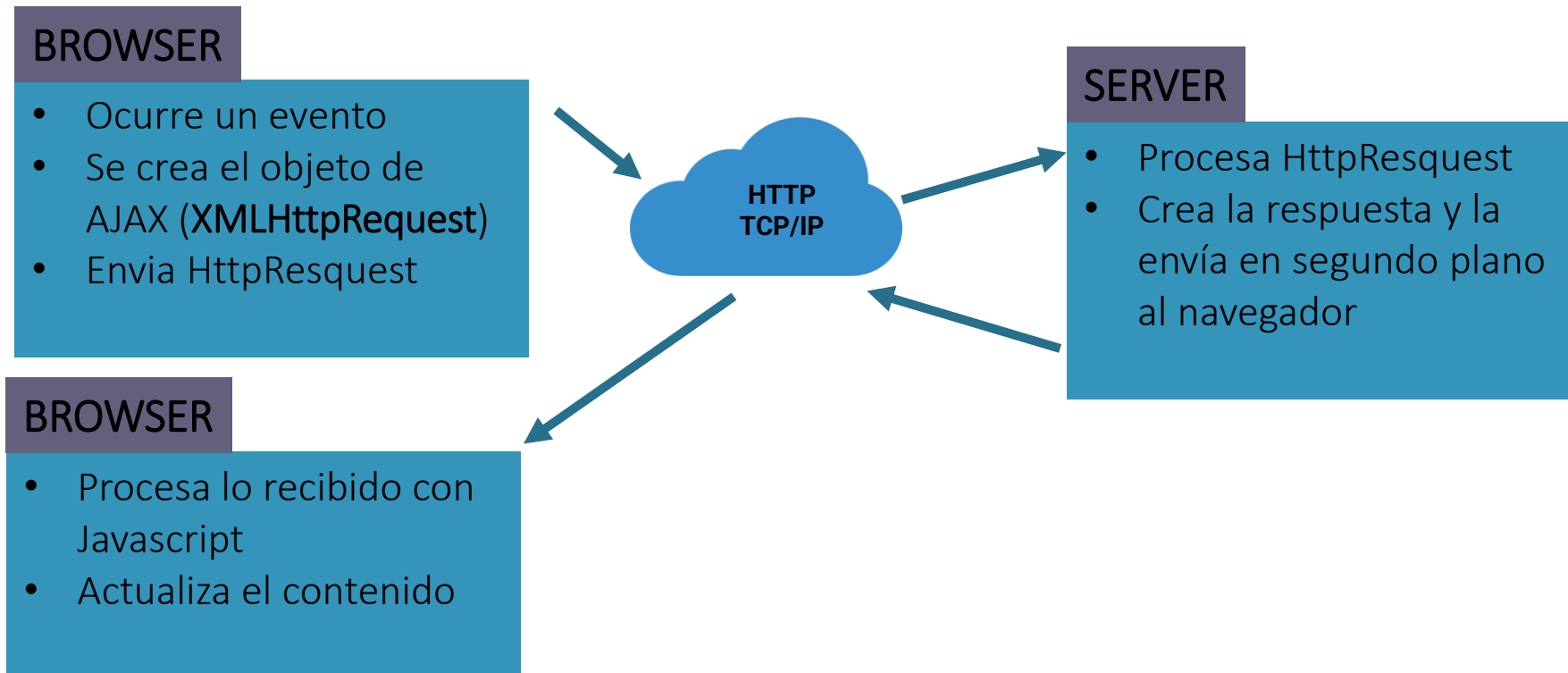


# Modelo de Aplicación clásico vs Ajax

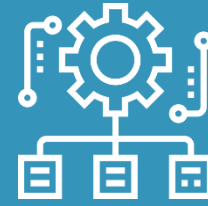




# AJAX – Como trabaja



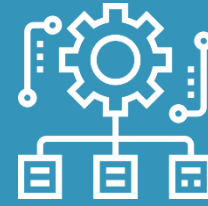
# AJAX - Modelo tradicional



En el **Modelo Tradicional**, las acciones del usuario en la página (pinchar en un botón, seleccionar un valor de una lista, etc.) desencadenan llamadas al servidor. Una vez procesada la petición del usuario, el servidor devuelve una nueva página HTML al navegador del usuario.

Al realizar peticiones continuas al servidor, el usuario debe esperar a que se recargue la página con los cambios solicitados.

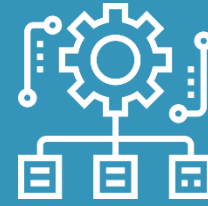
# AJAX – Modelo de Aplic.



**AJAX** mejora completamente la interacción del usuario con la aplicación, evitando las recargas constantes de la página, ya que el intercambio de información con el servidor se produce en un segundo plano.

Elimina la recarga constante de páginas mediante la creación de un elemento intermedio entre el usuario y el servidor. La nueva capa intermedia de AJAX mejora la respuesta de la aplicación, ya que el usuario nunca se encuentra con una ventana del navegador vacía esperando la respuesta del servidor.

# AJAX – Modelo de Aplic.

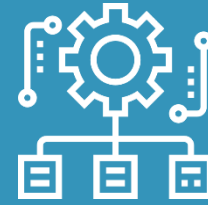


Las peticiones HTTP al servidor se sustituyen por peticiones JavaScript que se realizan al elemento encargado de AJAX.

Las peticiones más simples no requieren intervención del servidor, por lo que la respuesta es inmediata.

Si la interacción requiere una respuesta del servidor, la petición se realiza de forma asíncrona mediante AJAX. En este caso, la interacción del usuario tampoco se ve interrumpida por recargas de página o largas esperas por la respuesta del servidor.

# AJAX



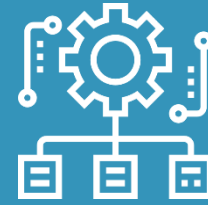
La historia de AJAX está íntimamente relacionada con un objeto de programación llamado **XMLHttpRequest**. El origen de este objeto se remonta al año 2000, con productos como Exchange 2000, Internet Explorer 5 y Outlook Web Access (OWA).

Durante el desarrollo de OWA, se evaluaron dos opciones:

- un cliente formado sólo por páginas HTML estáticas que se recargaban constantemente
- un cliente realizado completamente con HTML dinámico o DHTML.

Sin embargo, para ser realmente útil a esta última le faltaba un componente esencial: "algo" que evitara tener que enviar continuamente los formularios con datos al servidor. De aquí surge la primera versión de lo que denominó **XMLHTTP**

# AJAX - Consideraciones

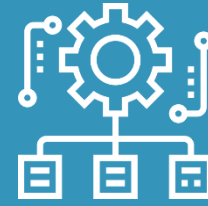


Las páginas creadas dinámicamente mediante peticiones sucesivas AJAX, no son registradas de forma automática en el historial del navegador, así que haciendo clic en el botón de "volver" del navegador, el usuario no será devuelto a un estado anterior de la página, en cambio puede volver a la última página que visitó. Soluciones incluyen el uso de IFrames invisible para desencadenar cambios en el historial del navegador .

Los motores de búsqueda no analizan JavaScript. La información en la página dinámica no se almacena en los registros del buscador. Exceptuando Google, que desde el 2011 sí indexa contenido Ajax y JavaScript (pero no es eficiente aún su indexación ya que puede tardar en hacerlo. <https://developers.google.com/search/docs/advanced/guidelines/ajax-faqs?hl=es>)

Hay problemas usando Ajax entre nombres de dominios, a esto se le conoce como Same Origin Policy o Política del Mismo Origen, lo cual es una medida de seguridad que puede ser solucionada con Cross-Origin Resource Sharing (CORS). (esta técnica es firmando los scripts pero la mayoría de los script no son firmados y además, Internet Explorer versión 11 e inferiores no soporta script firmados).

# AJAX - Consideraciones

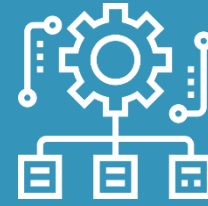


Además con los nombres de dominio, por razones de seguridad, los navegadores modernos no permiten el acceso a través de dominios . Esto significa que tanto la página web y el archivo XML que intenta cargar, deben estar ubicados en el mismo servidor (al menos que los se utilice CORS).

Dependiendo de como se desarrolle el sitio web, puedes mejorar o empeorar la carga en el servidor. Ajax puede ayudar al servidor a evitar la fase de renderización de HTML, dejándole ese trabajo al cliente, pero también puede sobrecargar al servidor si se hace varias llamadas a Ajax.

Es posible que páginas con Ajax no puedan funcionar en teléfonos móviles, PDA u otros aparatos. Ajax no es compatible con todos los software para ciegos u otras discapacidades.

# AJAX – Primera Aplicación



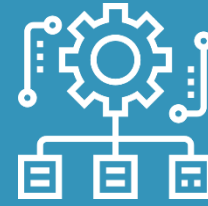
La aplicación AJAX completa más sencilla consiste en una adaptación del clásico *"Hola Mundo"*.

En este caso, una aplicación JavaScript descarga un archivo del servidor y muestra su contenido sin necesidad de recargar la página.

Holamundo.html



# AJAX – Primera Aplicación

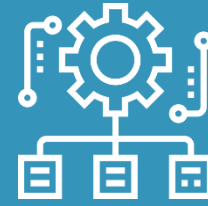


La aplicación AJAX del ejemplo anterior se compone de cuatro grandes bloques:

- instanciar el objeto XMLHttpRequest,
- preparar la función de respuesta,
- realizar la petición al servidor
- ejecutar la función de respuesta.

Todas las aplicaciones realizadas con técnicas de AJAX deben instanciar en primer lugar el objeto XMLHttpRequest, que es el objeto clave que permite realizar comunicaciones con el servidor en segundo plano, sin necesidad de recargar las páginas.

# AJAX – Primera Aplicación

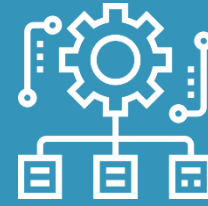


## objeto XMLHttpRequest

La implementación del objeto XMLHttpRequest depende de cada navegador

```
if(window.XMLHttpRequest) { // Navegadores que siguen los estándares  
    petición_http = new XMLHttpRequest();  
}  
  
else if(window.ActiveXObject) { // Navegadores obsoletos  
    petición_http = new  
        ActiveXObject("Microsoft.XMLHTTP");  
}
```

# La Primera Aplicación (cont)

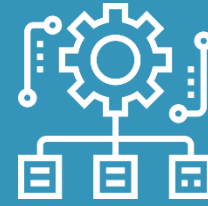


Una vez obtenida la instancia del objeto XMLHttpRequest, se prepara la función que se encarga de procesar la respuesta del servidor. Esta puede hacerse con dos propiedades **onreadystatechange** o **onload**.

La propiedad **onreadystatechange** del objeto XMLHttpRequest permite indicar esta función directamente incluyendo su código mediante una función anónima o indicando una referencia a una función independiente. Además esta propiedad nos permite conocer el estado de la llamada a Ajax. La propiedad **onload** solo contempla el estado cargado.

```
peticion_http.onreadystatechange = muestraContenido;
```

# La Primera Aplicación (cont)

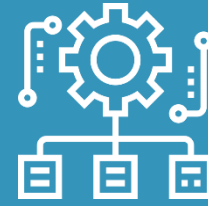


Después de preparar la aplicación para la respuesta del servidor, se realiza la petición HTTP al servidor:

```
peticion_http.open('GET', 'http://localhost/prueba.txt', true);  
peticion_http.send(null);
```

Las instrucciones anteriores realizan el tipo de petición más sencillo que se puede enviar al servidor. En concreto, se trata de una petición de tipo GET simple que no envía ningún parámetro al servidor. La petición HTTP se crea mediante el método `open()`, en el que se incluye el tipo de petición (GET), la URL solicitada (`http://localhost/prueba.txt`) y un tercer parámetro indica asíncrono: **True (asíncrono)** o **false (síncrona)**

# ¿Método Get o método Post?



## ¿Método Get o método Post?

GET es más simple y más rápido que POST, y se puede utilizar en la mayoría de los casos.

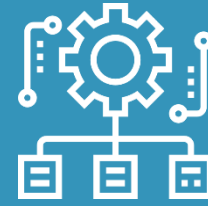
Sin embargo, siempre use las peticiones POST cuando:

- Debiera actualizar un archivo o una base de datos del lado del servidor

- Se envía de una gran cantidad de datos al servidor (POST no tiene limitaciones de tamaño).

- Envío de entrada del usuario (que puede contener caracteres desconocidos), POST es más robusto y seguro que el GET.

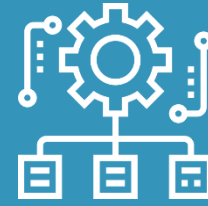
# Asíncrono ¿true o false?



Mediante el envío de forma asíncrona (true), el código JavaScript no tiene que esperar a la respuesta del servidor, sino que puede ejecutar otros scripts a la espera de respuesta del servidor.

El envío con False no es recomendable porque si el servidor es lento no se le permitirá seguir la ejecución del javascript hasta que la respuesta esté lista. Si el servidor tarda demasiado esta función se puede colgar o dejar de ejecutar.

# La Primera Aplicación (cont)



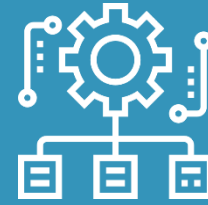
Por último, cuando se recibe la respuesta del servidor, la aplicación ejecuta de forma automática la función establecida anteriormente. (utilizando *onreadystatechange*)

```
function muestraContenido() {  
    if(peticion_http.readyState == 4) {  
        if(peticion_http.status == 200) {  
            alert(peticion_http.responseText);  
        }  
    }  
}
```

*Si usamos onload no consideramos los estados de la petición*

```
function muestraContenido() {  
    alert(peticion_http.responseText);  
}
```

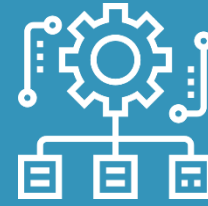
# AJAX – Propiedades del objeto



| Propiedad           | Descripción                                                                                                                                        |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>readyState</b>   | Valor numérico (entero) que almacena el estado de la petición                                                                                      |
| <b>responseText</b> | El contenido de la respuesta del servidor en forma de cadena de texto                                                                              |
| <b>responseXML</b>  | El contenido de la respuesta del servidor en formato XML. El objeto devuelto se puede procesar como un objeto DOM                                  |
| <b>status</b>       | El código de estado HTTP devuelto por el servidor (200 para una respuesta correcta, 404 para "No encontrado", 500 para un error de servidor, etc.) |
| <b>statusText</b>   | El código de estado HTTP devuelto por el servidor en forma de cadena de texto: "OK", "Not Found", "Internal Server Error", etc.                    |



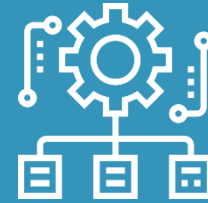
# AJAX – Objeto readyState



Valores para el Objeto **readyState**

| Valor | Descripción                                                                                               |
|-------|-----------------------------------------------------------------------------------------------------------|
| 0     | No inicializado (objeto creado, pero no se ha invocado el método <b>open</b> )                            |
| 1     | Cargando (objeto creado, pero no se ha invocado el método <b>send</b> )                                   |
| 2     | Cargado (se ha invocado el método <b>send</b> , pero el servidor aún no ha respondido)                    |
| 3     | Interactivo (se han recibido algunos datos, aunque no se puede emplear la propiedad <b>responseText</b> ) |
| 4     | Completo (se han recibido todos los datos de la respuesta del servidor)                                   |

# AJAX



Este ejemplo el AJAX modifica un DIV

```
<div id="demo">
```

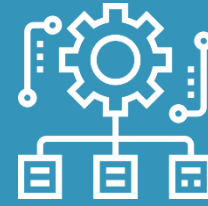
```
<h1>AJAX cambiando el contenido de una página</h1>
```

```
<button type="button" onclick="cargar_documento()">Cargar archivo  
holamundo.txt</button>
```

```
</div>
```

Holamundo2.html y  
holamundoonload.html

# AJAX – Ejemplo (cont)



Script del ejemplo el AJAX modifica un DIV

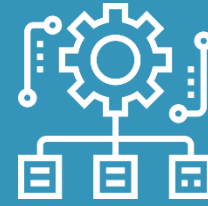
```
<script>
```

```
function cargar_documento() {  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {  
            document.getElementById("demo").innerHTML =  
                this.responseText;  
        }  
    };  
    xhttp.open("GET", "holamundo.txt", true);  
    xhttp.send();  
}
```

```
</script>
```

Holamundo2.html

# AJAX – Ejemplo (cont)



## Script del ejemplo el AJAX modifica un DIV

```
<script>
```

```
function loadDoc() {
```

```
    const xhttp = new XMLHttpRequest();
```

```
    xhttp.onload = function() {
```

```
        document.getElementById("demo").innerHTML = this.responseText;
```

```
    }
```

```
    xhttp.open("GET", "holamundo.txt");
```

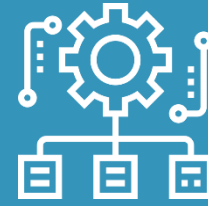
```
    xhttp.send();
```

```
}
```

```
</script>
```

Holamundoonload.html

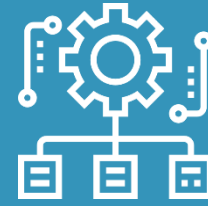
# AJAX – Ejemplo 1



Se desea hacer una página en donde desde el menú de opciones se carguen los contenidos de los links sin necesidad de cargar nuevamente el menú.

Para esto se construye la página index.html y se realizan las llamadas a los links por medio de javascript a las funciones del AJAX.

# AJAX – Ejemplo 1



```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml" lang="es" xml:lang="es">
```

```
<head>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
```

```
<title>Asincrono Javascript y XML</title>
```

```
<link href="estilos.css" rel="stylesheet" type="text/css">
```

```
<script language="JavaScript" type="text/javascript" src="js/codigo.js"></script>
```

```
</head>
```

```
<body>
```

```
<div id="menu">
```

```
<a href="javascript:llamadasin('ejemplos/ej1.html', 'contenido');">Ejemplo 1</a>
```

```
<a href="javascript:llamadasin('ejemplos/ej2.html', 'contenido');">Ejemplo 2</a></div>
```

```
<div id="contenido">
```

```
<h3 class="titulo">En este div se cargará asincronamente el contenido del ejemplo </h3>
```

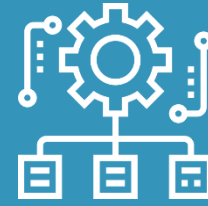
```
<p>Escoge una opción del menú;... </p>
```

```
</div>
```

```
</body>
```

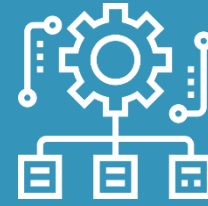
```
</html>
```

# AJAX – Ejemplo 1



```
function llamadasin(url, contenedor){  
    var pagina_requerida = false  
    if (window.XMLHttpRequest) {  
        // comprueba si el navegador es opera, safari, mozilla, etc.  
        pagina_requerida = new XMLHttpRequest()  
    }  
    else if (window.ActiveXObject){  
        // comprueba si el navegador es internet explorer  
        try {  
            pagina_requerida = new ActiveXObject("Msxml2.XMLHTTP")  
        }  
        catch (e){  
            // caso de versión antigua de internet explorer  
            try{  
                pagina_requerida = new ActiveXObject("Microsoft.XMLHTTP")  
            }  
        }  
    }  
}
```

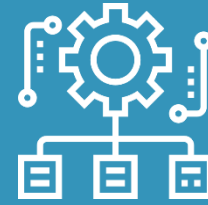
# AJAX - Ejemplo 1 (cont)



```
pagina_requerida.onreadystatechange=function(){  
    // llamada a la función que carga la página  
        pintapagina(pagina_requerida, contenedor)  
}  
// métodos open y send  
pagina_requerida.open('GET', url, true)  
pagina_requerida.send(null)  
}  
// función que presenta la información  
function pintapagina(pagina_requerida, contenedor){  
    if (pagina_requerida.readyState == 4 && (pagina_requerida.status==200 || window.location.href.indexOf("http")==-1))  
        document.getElementById(contenedor).innerHTML=pagina_requerida.responseText  
}
```



# AJAX CON FORMULARIOS



## ¿QUE SUCEDE CON LOS FORMULARIOS?

Tenemos que tomar cada campo del formulario o las variables a evaluar y las debemos concatenar de la forma “variable=valor”

```
function cargarContenido(){
var t1, t2, contenedor;
contenedor = document.getElementById('contenedor');
t1 = document.getElementById('texto1').value;
t2 = document.getElementById('texto2').value;

ajax=nuevoAjax();
ajax.open("POST", "ejemploajax2.php",true);

ajax.onreadystatechange=function() {
    if (ajax.readyState==4) {
        contenedor.innerHTML = ajax.responseText
    }
}
```

```
ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
```

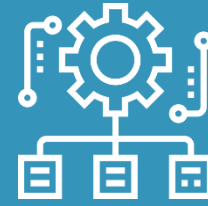
```
ajax.send("t1="+t1+"&t2="+t2)
}
```

Se puede enviar variables con el método **get** de la siguiente forma:

```
ajax.open("GET",
"ejemploajax2.php?VARIABLE=VALOR",true);
```

Se agregan variables con &variable\_valor

# AJAX – Validación



El siguiente caso es en donde queremos comprobar la disponibilidad de un campo en una base de datos, ejemplo el **nick name** de un usuario si necesidad de enviar el formulario para conocer si esta disponible o no.

Este ejemplo esta realizado con AJAX básico

`comprobar_nick_basico.php`

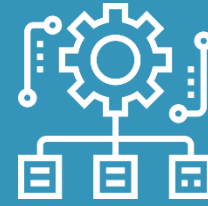
# Ejemplo 2 (validar nick en base de datos)



```
function comprobar_nick(str) {  
    if (str.length == 0) {  
        document.getElementById("mensaje").innerHTML = "";  
        return;  
    } else {  
        var xmlhttp = new XMLHttpRequest();  
        xmlhttp.onreadystatechange = function() {  
            if (this.readyState == 4 && this.status == 200) {  
                document.getElementById("mensaje").innerHTML = this.responseText;  
            }  
        };  
        xmlhttp.open("GET", "ajax_comprobar_nick.php?nickname=" + str, true);  
        xmlhttp.send();  
    }  
}
```

```
<p>  
    Complete Datos: </p>  
<form>  
    Primer Nombre:  
<input          type="text"  
onkeyup="comprobar_nick(t  
his.value)">  
  
<span id="mensaje"></span>  
</form>
```

# AJAX - Frameworks

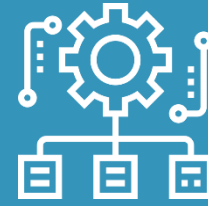


Existen muchos frameworks JavaScript que ya contemplan la instanciación y el uso de Ajax. PHP tiene su propio frameworks o librería denominada [XAJAX](#).

Existen otras como [prototype.js](#), [angular.js](#), [jquery.js](#) que también tienen sus métodos para el uso de Ajax.

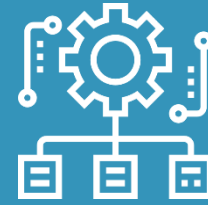
El uso de lo frameworks no condiciona a los desarrolladores a implementar sus propios mecanismos de AJAX u de otros frameworks.

# EJEMPLO 3 (prototype)



Para este ejemplo se utilizara el frameworks Prototype Javascript “[prototype.js](#)” que permite validar documentos, entre otras funciones. Es similar a [LightBox 2](#) pero hoy en día hay varias funciones similares para validar documentos.

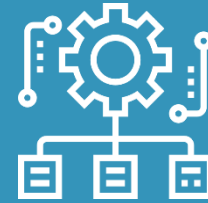
# Ejemplo 3 (pagina html)



Validamos el Nick en la base de datos usando Prototype

```
<html>
<head>
  <title>Enviar y procesar un formulario con Ajax y PHP</title>
  <script src="prototype.js" type="text/javascript"></script>
  <script type="text/javascript"><!--
    function comprobar(nick)
    {
      var url = 'http://' + location.host + '/ajax/ajax/ajax_comprobar_nick.php';
      var pars= ("nickname=" + nick);
      var myAjax = new Ajax.Updater( 'comprobar_mensaje', url, { method:
'get', parameters: pars});
    }
  // -->
</script>
</head>
```

# Ejemplo 3 (cont)



```
<body>

<h1>Recibir y procesar formulario con Ajax y PHP</h1>
<div id="mensaje">
<form id="formulario" name="formulario">
  Nickname<input name="nickname" onKeyUp="comprobar(this.value)" /> <span
  id="comprobar_mensaje"></span>
  <br>Nombre: <input type="text" name="nombre">
  <br>
  <br>Apellidos: <input type="text" name="apellidos">
  <br>
  <input type="button" value="Enviar"
  onclick="xajax_procesar_formulario(xajax.getFormValues('formulario'))">
</form>
</div>

</body>
</html>
```

VER COMO SERIA UN FORMULARIO ENVIADO CON AJAX EN `cargar_paginas.js` con la particularidad de ser adaptativo a varias paginas web con o sin parámetros y podemos tener un parámetro adicional al final

# Metodo moderno

Actualmente se usa un método moderno para realizar llamadas asíncronas en javascript basado en promesas

La interfaz de la **API Fetch** permite descargar (obtener) datos de un servidor web. Es mas sencillo que ``XMLHttpRequest()``

``await fetch()`` es la API moderna para obtener datos de un servidor HTTP.

``XMLHttpRequest()`` era la API heredada (antigua) para obtener datos de un servidor HTTP. ( **Actualmente Se usa solo para cuando es necesario tener un control estricto en la carga de información o muchas conexiones sincrónicas al mismo tiempo enlazadas**)



# Metodo moderno

## Descargar un archivo

```
// Fetch a file
fetch(file)
  .then(function(response) {
    return response.text();
  })
  .then(function(data) {
    myDisplayer(data);
  });
```

**O Usando flechas**

```
// Fetch a file
fetch(file)
  .then(response => response.text())
  .then(data => myDisplayer(data));
```

fetch1.html  
Fetch2.html

# Metodo moderno

## Usando una función asincrónica

- La API Fetch permite a JavaScript descargar recursos desde un servidor web.
- Se puede utilizar para cargar archivos de texto, datos JSON, imágenes y muchos otros tipos de recursos.
- La descarga es asíncrona.
- JavaScript puede seguir ejecutándose mientras espera la respuesta del servidor.
- El método `fetch()` devuelve una promesa que se resuelve en un objeto `Response`.

```
async function loadText(file) {  
  const response = await fetch(file);  
  myDisplayer(await response.text());  
}
```

```
// llamda a la funcion  
loadText("fetch.txt");
```

fetch3.html

# Metodo moderno

fetch() devuelve una promesa que se resuelve en un objeto Response.

| Property / Method          | Description                          |
|----------------------------|--------------------------------------|
| <code>ok</code>            | True if the request succeeded.       |
| <code>status</code>        | The HTTP status code.                |
| <code>statusText</code>    | The HTTP status message.             |
| <code>text()</code>        | Reads the response as text.          |
| <code>json()</code>        | Reads the response as JSON.          |
| <code>blob()</code>        | Reads the response as binary data.   |
| <code>bytes()</code>       | Reads the response as bytes.         |
| <code>arrayBuffer()</code> | Reads the response as an ArrayBuffer |

# Metodo moderno

fetch() devuelve una promesa que se resuelve en un objeto Response.

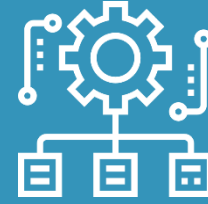
## Ejemplo ok

```
async function loadText(file) {  
  const response = await fetch(file);  
  myDisplayer(response.ok);  
}
```

## Ejemplo Json

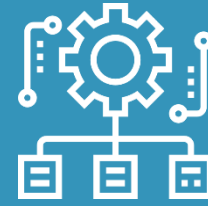
```
async function loadCustomer() {  
  const response = await fetch("customer.json");  
  const customer = await response.json();  
  myDisplayer(customer.name);  
}  
  
loadCustomer();
```

# Bibliografía



Prototype.js : <http://www.anieto2k.com/2006/12/18/tutorial-de-ajax-con-prototype/>  
(api) <http://www.prototypejs.org/learn/introduction-to-ajax>

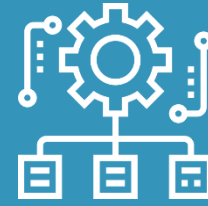
# Bibliografía



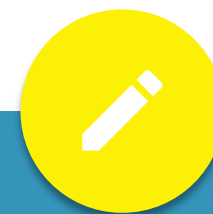
Librería xajax para php: <http://www.desarrolloweb.com/manuales/xajax.html>

Sitio oficial <http://xajaxproject.org>

# Bibliografía



Javier Eguíluz Pérez: Introducción a Ajax (pdf)



Programación Orientada a la Web, Ago. – Dic. 2026